

Name: Godwin M. Labaya

Course & Section: BSIT – 3 (IT-342)

1. GitHub Repository Link

<https://github.com/godwinlabaya/IT342-Labaya-DisasterAidConnect.git>

2. Final Commit for Phase 1

commit f2bc3daf83d2d07027f95dc9d40691d80028d359 (HEAD -> main, origin/main, origin/HEAD)

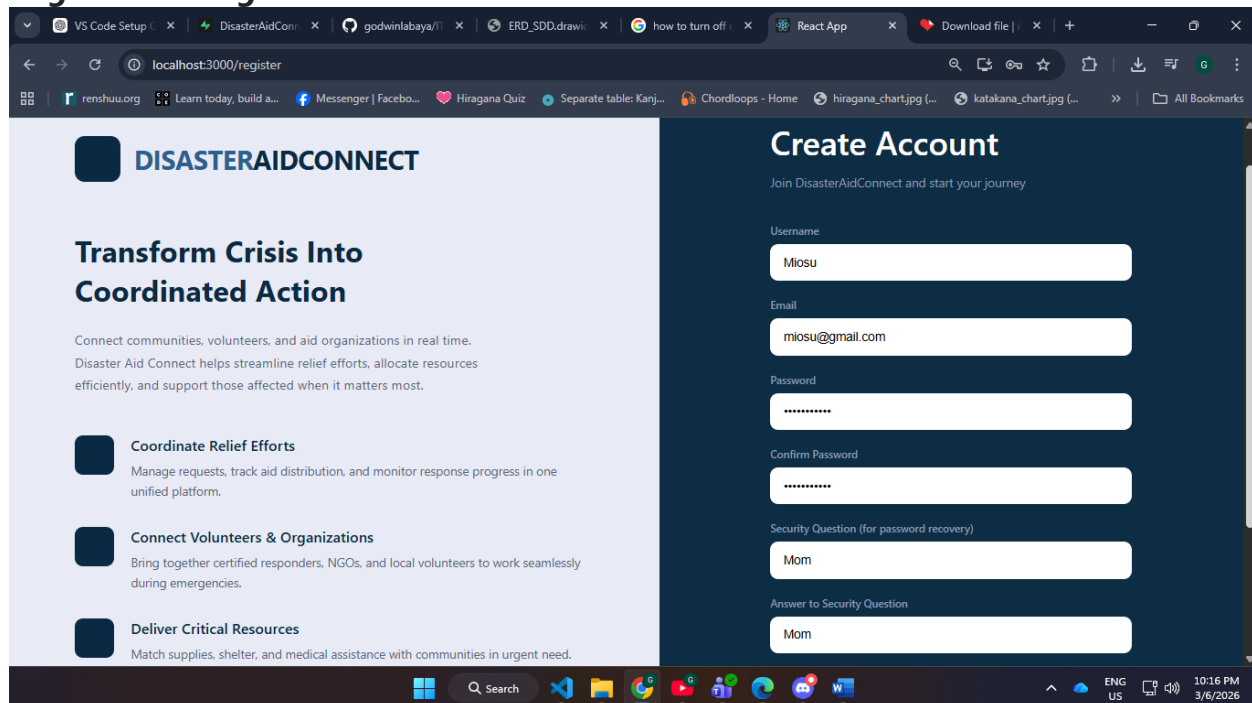
Author: LAB24_USER ON WS21 <L24X09W21@LabHCI.net>

Date: Fri Mar 6 09:48:44 2026 +0800

IT342 Phase 1 – User Registration and Login Completed

3. Screenshots of Implementation

Registration Page



DISASTERAIDCONNECT

Transform Crisis Into Coordinated Action

Connect communities, volunteers, and aid organizations in real time. Disaster Aid Connect helps streamline relief efforts, allocate resources efficiently, and support those affected when it matters most.

- Coordinate Relief Efforts**
Manage requests, track aid distribution, and monitor response progress in one unified platform.
- Connect Volunteers & Organizations**
Bring together certified responders, NGOs, and local volunteers to work seamlessly during emergencies.
- Deliver Critical Resources**
Match supplies, shelter, and medical assistance with communities in urgent need.

Create Account
Join DisasterAidConnect and start your journey

Username
Miosu

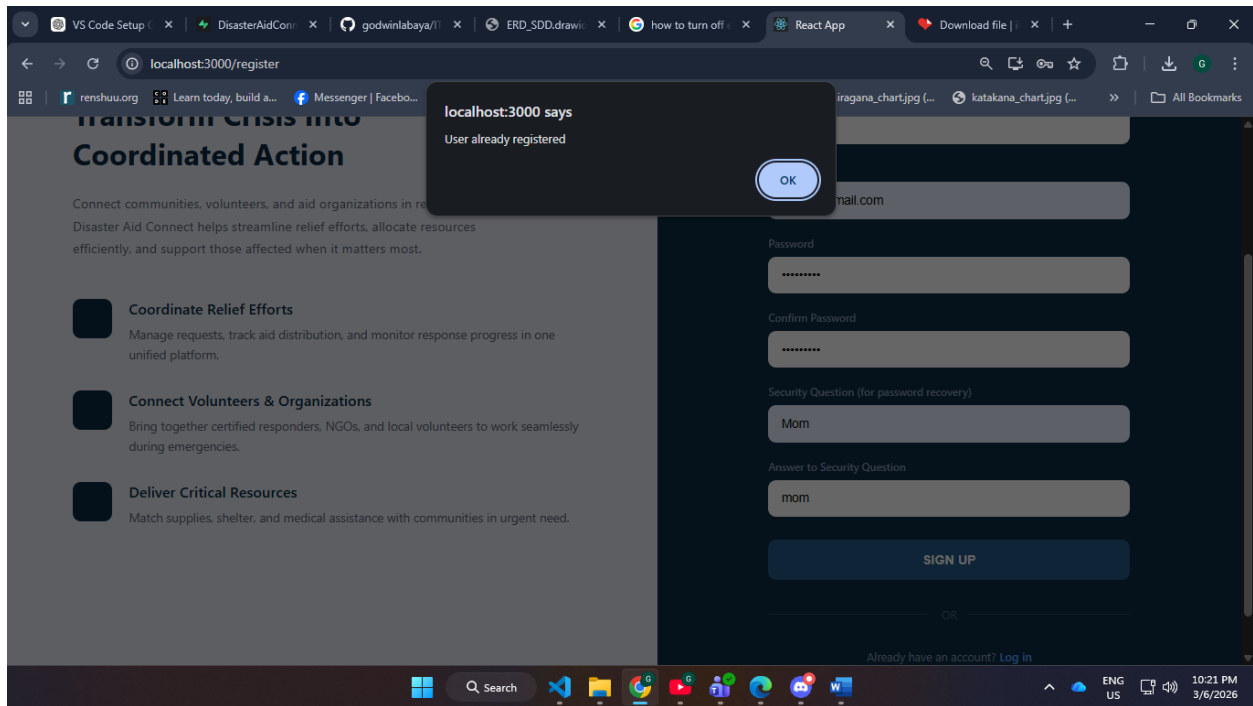
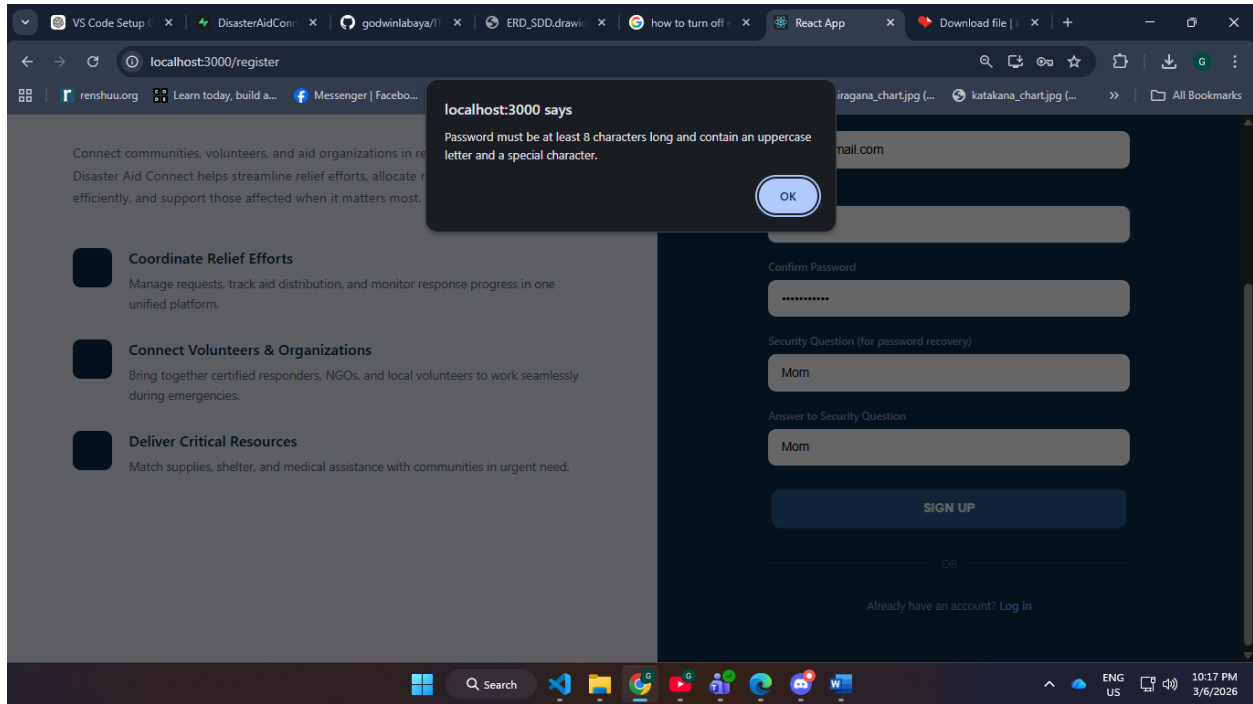
Email
miosu@gmail.com

Password

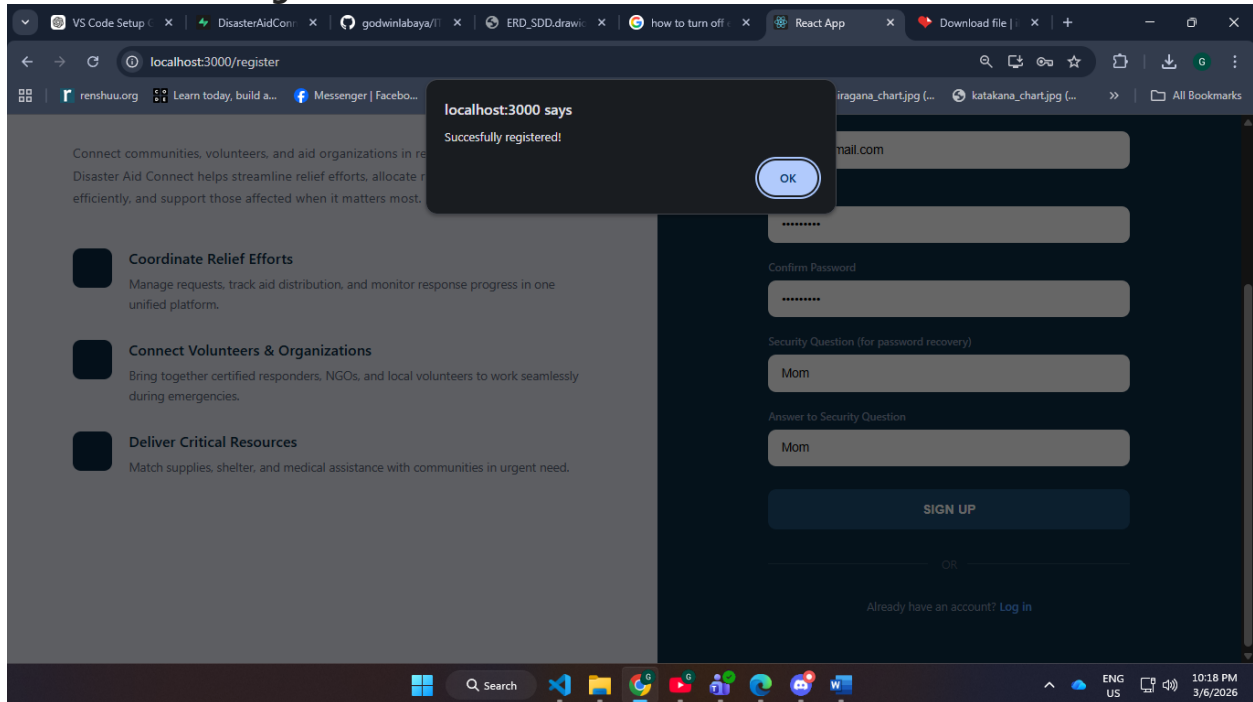
Confirm Password

Security Question (for password recovery)
Mom

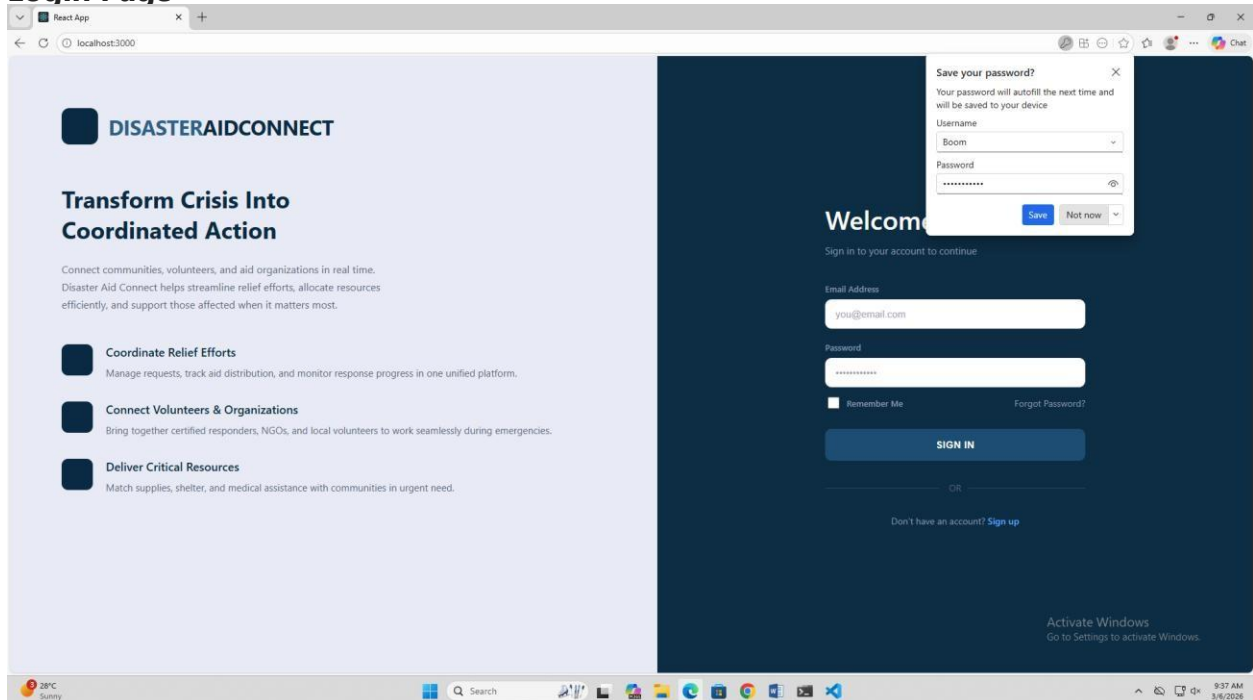
Answer to Security Question
Mom



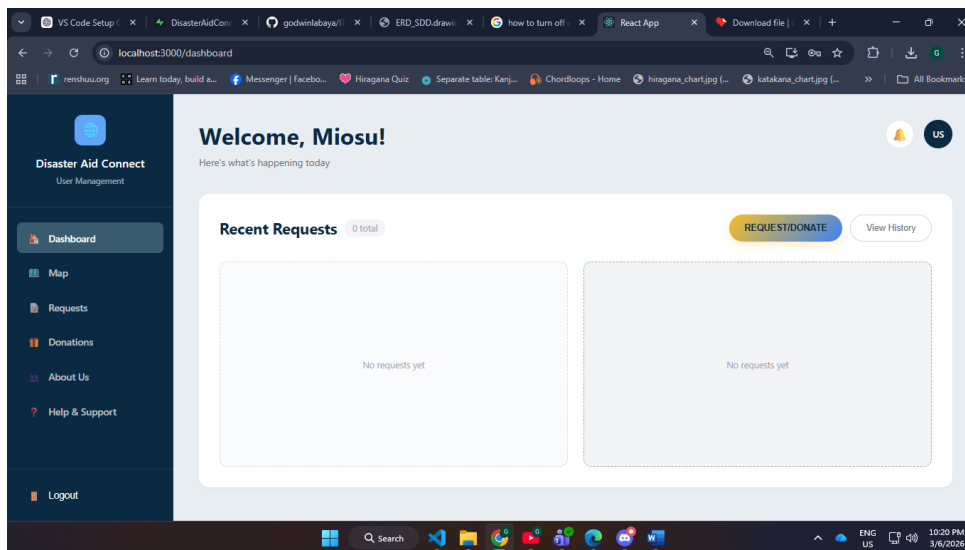
Successful user registration



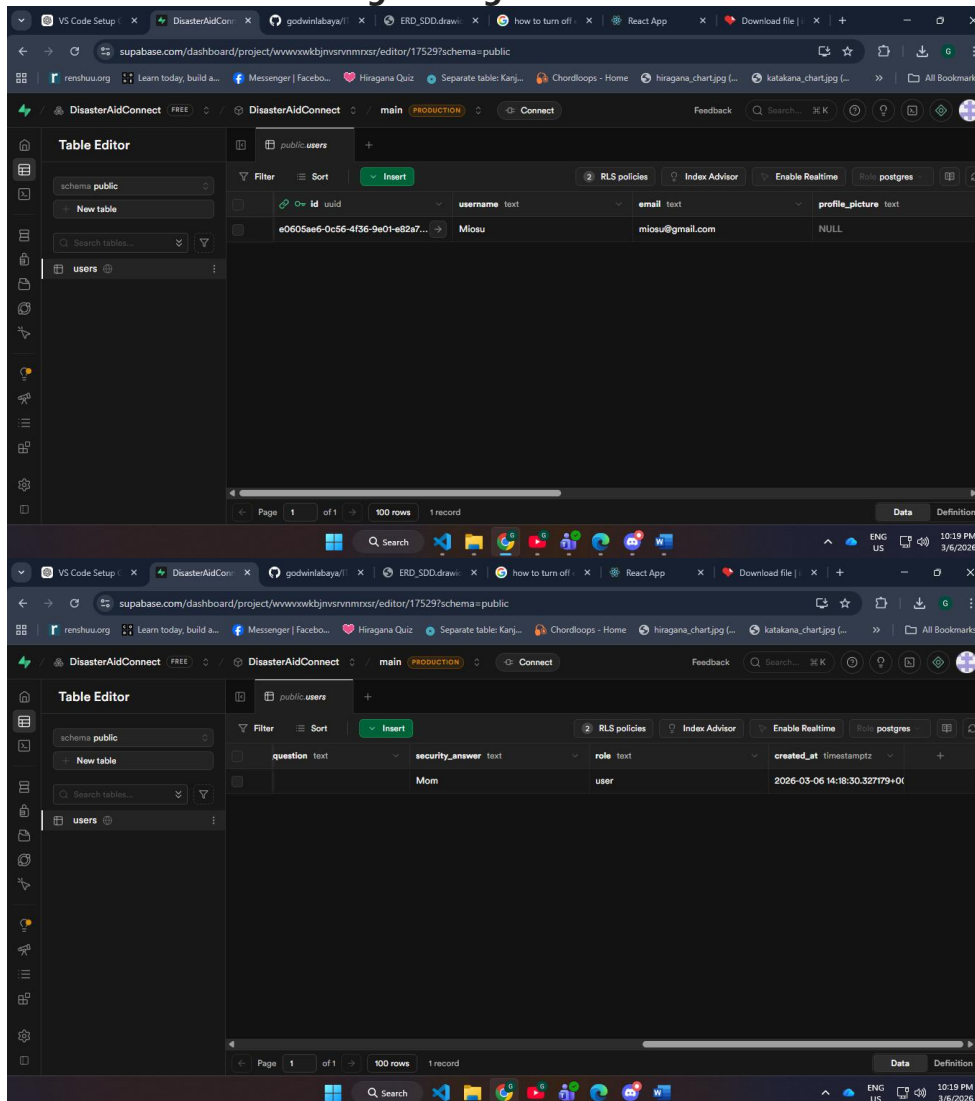
Login Page



Successful login



Database record showing the registered user



4. Short Implementation Summary (1 Page)

User Registration

- **Registration fields used:**
 - `username` – User's display name
 - `email` – User's email address
 - `password` – Password for account login
 - `confirmPassword` – To verify password correctness
 - `securityQuestion` – Question for password recovery
 - `securityAnswer` – Answer to the security question
- **Validation process:**
 - Checks if `password` matches `confirmPassword`.
 - Checks if `password` has 1 capital letter, 1 special character and a minimum of 8 total characters
 - Supabase automatically validates the email format.
- **How duplicate accounts are prevented:**
 - Supabase authentication service ensures each email is unique.
 - Attempting to register an existing email triggers an error message.
- **How passwords are stored securely:**
 - Supabase handles password hashing internally using secure hashing algorithms (bcrypt).
 - The app never stores plain text passwords.

User Login

- **Login credentials used:**
 - Email and password.
- **How the system verifies users:**
 - Checks if email and password exists in the Supabase database
 - Supabase compares the hashed password stored in the database with the entered password.
- **What happens after successful login:**
 - User is authenticated and navigated to the main application/dashboard.
 - Authentication session is managed by Supabase, allowing secure access to protected routes.

Database Table

- **Table:** `users`
- **Columns:**
 - `id` – Supabase `uuid` for user identification
 - `email` – Email display
 - `username` – User display name
 - `profile_picture` – for profile picture
 - `profile_picture_mime_type` – for profile picture format
 - `security_question` – Password recovery question
 - `security_answer` – Password recovery answer
 - `role` – User role (e.g., `user`)
- **Notes:**
 - Supabase also maintains its internal `auth.users` table that stores emails, hashed passwords, and metadata.

API Endpoints (if applicable)

- **Register user:**
`POST /api/auth/register` → Handled via `Supabase.auth.signUp()`
- **Login user:**
`POST /api/auth/login` → Handled via `Supabase.auth.signInWithPassword()`